

CONTENT ENGINE V2.0 — REDESIGN DOCUMENT

“Build, Operate, Scale, Own the Failure”

A. PRODUCT MODEL

A.1. Đây là sản phẩm gì?

Content Workspace with AI Copilot

Không phải chatbot. Không phải assistant. Không phải agent playground.

Đây là **workspace** — nơi user tạo, chỉnh sửa, và xuất bản marketing content. AI là **copilot** — ngồi cạnh user, hiểu ý, gợi ý, thực hiện khi được giao.

Tại sao chọn “workspace” thay vì “chatbot”?

Tiêu chí	Chatbot	Workspace
User expectation	AI hiểu mọi thứ	AI hỗ trợ, user kiểm soát
Error tolerance	Thấp —1 lần sai = bỏ	Cao —user thấy và sửa được
Learning curve	Ẩn (phải học “nói đúng”)	Hiển rõ (UI familiar)
Revenue model	Khó monetize	Per-seat, per-output dễ tính
Debug	Khó —state invisible	Dễ —mọi thứ visible

Mental model cho user:

Tôi có một dự án marketing.
Tôi mở workspace.
Tôi nói với AI copilot: "Làm bài Facebook cho sản phẩm X".
AI tạo draft.
Tôi sửa trực tiếp trên draft.
AI học từ sửa của tôi.
Tôi duyệt và đăng.

B. CORE USER JOURNEY

B.1. Journey Map (ít ma sát)

[1] LANDING
↓
User đăng ký / đăng nhập
↓
[2] ONBOARDING (3 phút, không bỏ qua được)
↓
AI hỏi 3 câu:
- "Bạn làm ngành gì?" (dropdown + tự nhập)

- "Thương hiệu của bạn tên gì?" (text)
- "Bạn thường đăng kênh nào?" (multi-select: Facebook, Instagram, Blog, TikTok)

↓

AI tạo "Brand Profile" — lưu vào memory, hiển thị cho user xem.
User có thể sửa ngay.

↓

[3] DASHBOARD

↓

Hiển thị:

- Các dự án gần đây
- Nút "Tạo content mới" (prominent)
- Credit còn lại

↓

[4] CREATE SESSION

↓

User click "Tạo content mới"

↓

AI hiển thị prompt input (giống chat nhưng là 1 input box lớn)
Gợi ý placeholder: "VD: Viết bài Facebook quảng cáo bánh mì thịt nướng cho quán ABC"

↓

User nhập yêu cầu → ENTER

↓

[5] CLARIFICATION (nếu cần)

↓

AI phân tích yêu cầu.
Nếu thiếu thông tin quan trọng:

- Hiển thị 1-2 câu hỏi ngắn (không quá 3)
- Mỗi câu có suggested answer
- User chọn hoặc nhập → tiếp tục

↓

Nếu đủ thông tin → skip

↓

[6] GENERATION

↓

AI hiển thị "Đang tạo..." với progress bar
(không phải "typing..." vô tận)

↓

AI tạo output:

- Nếu đơn giản (1 caption): hiển thị ngay
- Nếu phức tạp (full campaign): hiển thị outline trước, user duyệt outline rồi mới generate chi tiết

↓

[7] REVIEW & EDIT

↓

Output hiển thị trong editor WYSIWYG
User có thể:

- Sửa trực tiếp (giống Google Docs)
- Chọn đoạn text → click "AI sửa" → chọn: "ngắn gọn hơn", "vui hơn", "thêm CTA", "viết lại"
- So sánh với bản cũ (diff view)
- Thêm comment

↓

Mỗi lần AI sửa → lưu version mới

```
User có thể undo/redo giữa các version
↓
[8] APPROVE
↓
User click "Duyệt" hoặc "Đăng"
↓
Nếu chưa connect kênh → hiển thị "Kết nối [kênh] trước"
↓
[9] PUBLISH
↓
AI gọi API kênh tương ứng
Hiển thị status: "Đang đăng..." → "Đã đăng" / "Lỗi: [chi tiết]"
↓
Nếu lỗi → hiển thị lỗi + nút "Thử lại" / "Đăng tay"
↓
[10] POST-PUBLISH
↓
Lưu vào history
Hiển thị "Bài đã đăng" với link
Hỏi: "Bạn muốn làm gì tiếp?" (gợi ý: tạo bài mới, xem analytics, sửa bài)
↓
Trừ credit
↓
[11] RETURN
↓
User quay lại dashboard
Các bài cũ hiển thị trong list
Click vào bài cũ → mở lại editor → sửa → đăng lại
```

B.2. Key Design Decisions

Decision	Lý do
Workspace thay vì chat	User cần thấy output, sửa trực tiếp, không bị surprise
Onboarding bắt buộc	Brand profile là prerequisite cho mọi output chất lượng. Không có = generic content = user bỏ
Clarification tối đa 2 lượt	Tránh chat vô tận. Nếu sau 2 lượt vẫn thiếu → AI tự chọn mặc định + ghi chú
Outline trước cho campaign phức tạp	Giảm waste. User duyệt outline (free) trước khi generate chi tiết (tốn credit)
Editor WYSIWYG + AI edit inline	User sửa theo cách quen thuộc. AI học từ edit pattern
Version control	User không sợ “AI sửa hỏng”. Có thể quay lại bất kỳ version nào
Credit trừ sau publish	Không trừ khi user chỉ preview/sửa. Fair.

C. CONVERSATION DESIGN

C.1. AI không phải chatbot. AI là copilot trong workspace.

Nguyên tắc giao tiếp:

Tình huống	AI làm gì	AI không làm gì
User nhập yêu cầu	Phân tích, hỏi clarification nếu cần	Không tự động generate ngay nếu thiếu info
User sửa content	Ghi nhận pattern, apply lần sau	Không hỏi “bạn có chắc không”
User duyệt	Đăng ngay	Không hỏi lại
User hủy	Lưu draft, không trừ credit	Không giữ giận
User hỏi thông tin	Trả lời ngắn gọn, đúng trọng tâm	Không gợi ý “có muốn làm bài không”

C.2. Khi nào hỏi thêm?

AI hỏi thêm khi VÀ CHỈ KHI:

- Thiếu thông tin **bắt buộc** để tạo output chất lượng:
 - Không biết sản phẩm/dịch vụ là gì
 - Không biết kênh đăng
 - Không biết mục tiêu (bán hàng / brand awareness / engagement)
- Yêu cầu mơ hồ có thể dẫn đến 3+ kết quả khác nhau:
 - “Làm bài hay” → hỏi: “Hay theo tiêu chí nào? HÀi hước / thông tin / cảm xúc?”
- Không hỏi khi:
 - Có thể dùng Brand Profile
 - Có thể dùng lịch sử từ memory
 - Có thể chọn mặc định hợp lý + ghi chú

C.3. Khi nào tự làm?

AI tự generate khi:

- Đã đủ thông tin (từ input + Brand Profile + memory)
- Yêu cầu rõ ràng (“viết caption Facebook cho bánh mì ABC”)
- Không có ambiguity

C.4. Khi nào chờ?

AI chờ khi:

- User đang sửa (editor mode)
- User chưa duyệt outline (campaign phức tạp)
- Đang trong clarification loop

C.5. Khi nào gợi ý?

AI gợi ý khi:

- User mở dashboard lần đầu trong ngày → “Hôm nay là [ngày lễ], có muốn làm bài không?”
- User vừa publish → “Bạn có muốn tạo variant cho Instagram không?”
- User đang xem bài cũ → “Bài này đã 30 ngày, có muốn refresh không?”

Quy tắc: Gợi ý 1 lần/session. Không spam.

C.6. AI học user như thế nào?

Không phải:

- Tự động extract từ chat
- Suy luận “user thích gì”
- Ghi nhận mọi thứ user nói

Mà là:

```
Explicit Signals (user chủ động cho biết):
├─ User sửa content → AI ghi: "edit: tone from formal → casual"
├─ User click "Đúng" / "Sai" trên suggestion → AI ghi preference
├─ User chọn "Lưu làm template" → AI ghi template
└─ User rate output 1-5 sao → AI ghi rating + reason

Implicit Signals (AI quan sát, không tự ghi):
├─ User thường sửa phần nào → pattern, không ghi vào profile
├─ User bỏ giữa chừng ở bước nào → analytics, không ghi vào profile
└─ User thường dùng kênh nào → analytics, không ghi vào profile
```

Memory lifecycle:

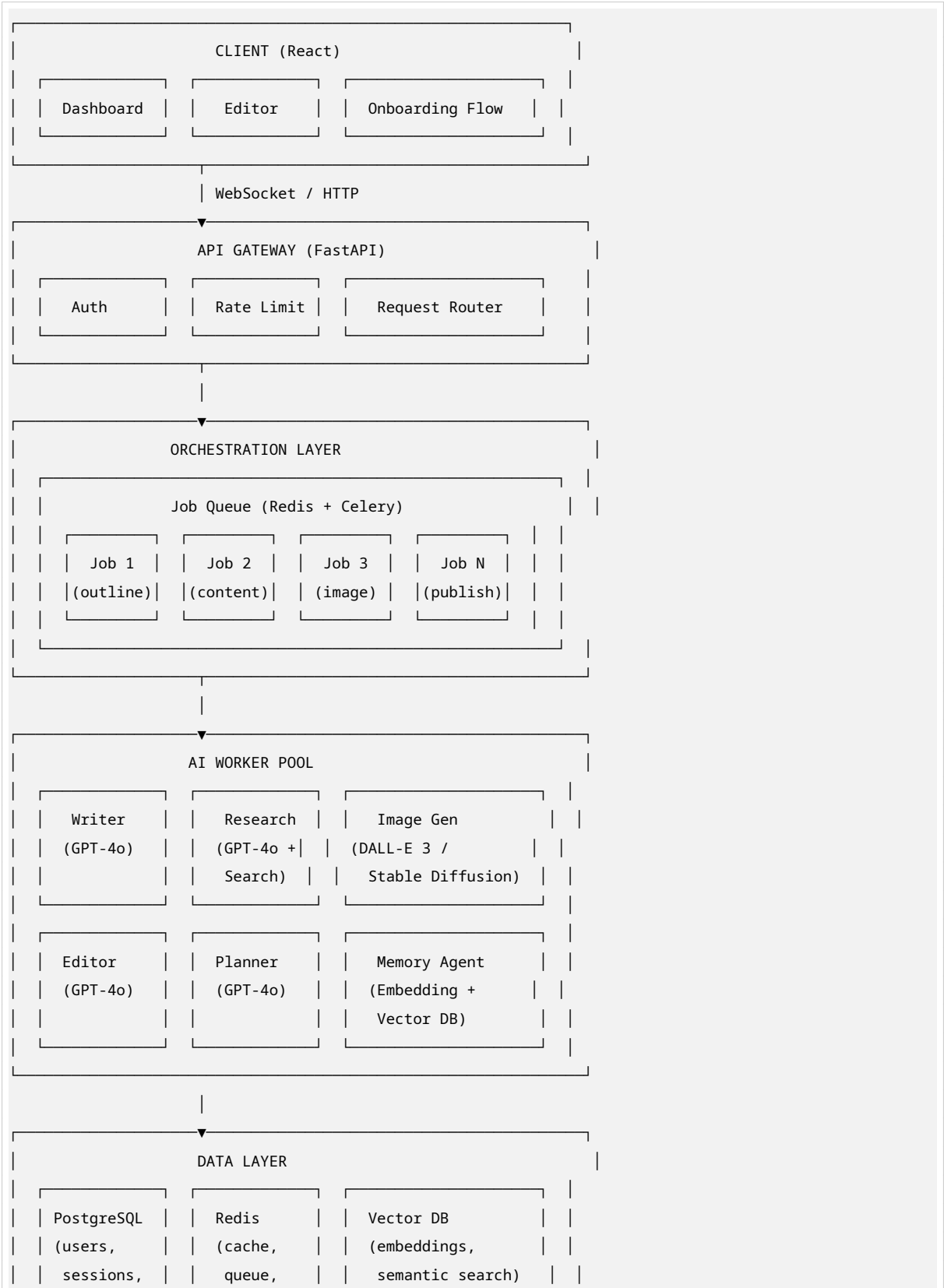
```
Short-term (session):
├─ Current draft content
├─ Edit history trong session
├─ Clarification context
└─ TTL: 24h sau khi session end

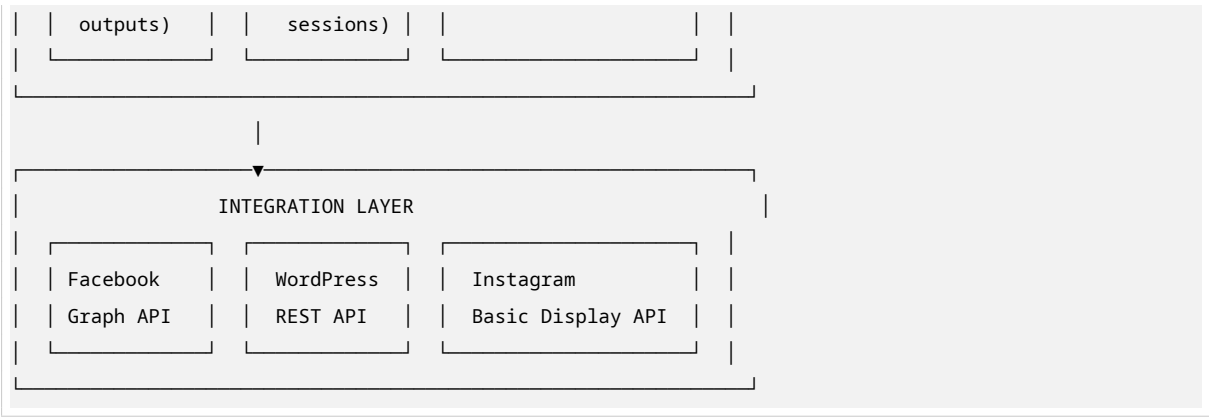
Long-term (persistent):
├─ Brand Profile (user chỉnh sửa, AI không tự ghi)
├─ Templates (user lưu)
├─ Published outputs (lịch sử)
├─ Explicit feedback (rating, edit notes)
└─ Preferences (user chọn trong settings)

Feedback Loop:
├─ User sửa → AI detect diff → categorize change → ask user: "Lần sau tôi nên viết thể này luôn?"
├─ User chọn "Yes" → ghi vào Preference
├─ User chọn "No, chỉ lần này" → ghi vào session memory only
└─ User không chọn → không ghi (tránh data pollution)
```

D. EXECUTION ENGINE

D.1. Kiến trúc tổng thể





D.2. Tại sao cần Orchestration Layer (Job Queue)?

Lý do	Giải thích
Non-blocking	User không chờ AI generate. Submit job → nhận notification khi xong
Retry	Job fail → tự động retry với exponential backoff
Parallel	Nhiều agent chạy song song (research + image + writing)
Cost control	Queue cho phép batch processing, rate limiting
Observability	Mỗi job có log, metrics, tracing riêng

D.3. Tại sao KHÔNG dùng “gọi thợ đúng việc” dynamic routing?

Quyết định: Dùng predefined workflow templates thay vì dynamic agent routing.

Dynamic Routing	Predefined Templates
AI phải plan workflow mỗi lần	Workflow cố định theo content type
Complexity cao, dễ fail	Predictable, dễ debug
Khó test, khó optimize	Mỗi template có thể benchmark riêng
Over-engineering cho MVP	Đủ cho 80% use case

Workflow Templates:

```
Template: "Social Post"
├─ Input: product, channel, tone
├─ Step 1: Writer → caption
├─ Step 2: Image Gen (optional) → image
├─ Step 3: Review → user edit
└─ Step 4: Publish → API call

Template: "Blog Post"
├─ Input: topic, keywords, length
├─ Step 1: Research → outline (user duyệt)
├─ Step 2: Writer → draft from approved outline
├─ Step 3: Editor → polish
└─ Step 4: Review → user edit
```

```
└─ Step 5: Publish → WordPress API

Template: "Full Campaign"
└─ Input: campaign theme, channels, budget
└─ Step 1: Research → insights
└─ Step 2: Planner → campaign structure (user duyệt)
└─ Step 3: [parallel]
|   └─ Writer → blog
|   └─ Writer → ads
|   └─ Writer → social captions
|   └─ Image Gen → campaign visuals
└─ Step 4: Review → user edit từng phần
└─ Step 5: Publish → từng kênh riêng
```

Mở rộng:

- Thêm template mới = thêm file config + 1 worker mới
- Không sửa orchestration layer
- Không sửa existing templates

D.4. Agent Design

Writer Agent:

```
Input: request + Brand Profile + context (lịch sử, templates)
Output: content draft
Model: GPT-4o (high quality, cost acceptable for output)
Prompt strategy:
- System prompt: Brand voice, constraints, format
- Few-shot: 3 examples from user's best-rated outputs
- Temperature: 0.7 (creative but controlled)
```

Research Agent:

```
Input: topic + date range
Output: structured insights (trends, competitors, keywords)
Model: GPT-4o + web_search tool
Cache: Research results cached 24h per topic
Cost optimization: Research chỉ chạy khi user explicitly yêu cầu hoặc template cần
```

Image Agent:

```
Input: prompt + style reference
Output: image URL
Model: DALL-E 3 API (không self-host)
Reason: 4GB VRAM không đủ cho SDXL. Self-host = maintenance hell.
Fallback: Nếu DALL-E down → hiển thị "Image generation temporarily unavailable"
```

Editor Agent:

```
Input: content + edit instruction ("ngắn gọn hơn", "thêm CTA")
```


Output: edited content
Model: GPT-4o
Strategy: Edit-in-place, không regenerate toàn bộ

- Parse content thành sections
- Identify section cần sửa
- Rewrite section đó, giữ nguyên context

Planner Agent (chỉ cho campaign phức tạp):

Input: campaign requirements
Output: structured outline
Model: GPT-4o
Purpose: User duyệt outline trước khi tốn credit generate chi tiết

D.5. Queue & Retry Design

```
# Pseudocode for job processing

class ContentJob:
    job_id: UUID
    user_id: UUID
    template: str # "social_post", "blog_post", "campaign"
    status: "pending" | "running" | "completed" | "failed" | "cancelled"
    steps: List[Step]
    current_step: int
    retry_count: int = 0
    max_retries: int = 3
    cost_estimate: float # tính trước khi chạy

    def execute(self):
        for step in self.steps[self.current_step:]:
            try:
                result = step.run()
                step.status = "completed"
                step.output = result
                self.current_step += 1
            except Exception as e:
                if self.retry_count < self.max_retries:
                    self.retry_count += 1
                    self.status = "retrying"
                    retry_with_backoff(self)
                else:
                    self.status = "failed"
                    notify_user(self.user_id, f"Step {step.name} failed: {e}")
                    break

        if self.current_step == len(self.steps):
            self.status = "completed"
            notify_user(self.user_id, "Content ready for review")
```

E. MEMORY DESIGN

E.1. Phân loại memory

MEMORY SYSTEM TIER 1: BRAND PROFILE (User-managed) brand_name industry products: [{name, description, price, target}] tone: "professional" "casual" "humorous" ... channels: ["facebook", "instagram", "blog"] image_style: "minimalist" "vibrant" "elegant" brand_colors: ["#FF0000", "#00FF00"] golden_hours: ["07:00", "19:00"] templates: [{name, content, channel}] created_at, updated_at → User chỉnh sửa trực tiếp. AI chỉ đọc, không tự ghi. → UI: Settings page với form rõ ràng.
TIER 2: SESSION MEMORY (Short-term, TTL 24h) current_draft: content + metadata edit_history: [version_1, version_2, ...] clarification_context: {questions, answers} user_intent: parsed từ input → Lưu trong Redis. TTL 24h sau session end. → Dùng cho context trong 1 phiên làm việc.
TIER 3: OUTPUT HISTORY (Persistent) outputs: [{output_id, content, channel, rating, ...}] campaigns: [{campaign_id, outputs[], metrics}] usage: [{date, tokens_used, cost}] → Lưu trong PostgreSQL. → Dùng cho "xem lại bài cũ", analytics.
TIER 4: LEARNED PREFERENCES (Explicit only) content_preferences: [{type, value, source}] Ví dụ: - {type: "tone", value: "casual", source: "user_edit"}

```
|   - {type: "cta_style", value: "soft", source: "rating"}
|   └─ rejected_patterns: [{pattern, reason, count}]
|   └─ template_suggestions: [{base_template, user_variant}] |
|
|
|   → Chỉ ghi khi user explicitly confirm.
|   → Mỗi preference có confidence score (1-5).
|   → Preference với score < 3 không dùng. |
```

E.2. Tại sao KHÔNG để AI tự học từ hành vi?

Vấn đề	Giải pháp mới
AI suy luận sai → profile bị bẩn	AI không tự ghi profile. Chỉ user chỉnh sửa.
“User hay đăng 7h sáng” = correlation, không phải causation	Analytics riêng, không ghi vào profile
Feedback loop dương	Explicit confirmation trước khi ghi preference
Hallucination ở memory	Memory chỉ lưu facts (output, rating), không lưu inference

E.3. How AI learns from edits

```
User sửa content trong editor
↓
AI detect diff (old vs new)
↓
AI categorize change:
- Tone change (formal → casual)
- Length change (long → short)
- Structure change (add CTA, remove paragraph)
- Fact correction
↓
AI hiển thị: "Tôi thấy bạn thường sửa [tone] thành [casual]. Lần sau tôi nên viết thế này luôn?"
↓
User chọn:
- "Có, luôn viết casual" → ghi vào Preference (score: 5)
- "Chỉ lần này" → không ghi
- "Không, đây là ngoại lệ" → ghi vào Rejected Patterns
↓
Preference được dùng trong prompt của lần sau
```

F. REVIEW & EDIT SYSTEM

F.1. Editor Design

AI CONTENT EDITOR

Version: v3

Compare v2

Compare v1

Bánh mì thịt nướng ABC

Hôm nay quán có món mới...

Chọn đoạn text → AI sửa ▼

Ngắn gọn hơn

Vui hơn

Thêm CTA

Viết lại đoạn này

Giống bài cũ [list bài cũ]

Thêm comment

✖ Hủy

Lưu draft

✔

Duyệt & Đăng

F.2. Edit Operations

Thao tác	Cách làm	AI học được gì
Sửa trực tiếp	User gõ như Google Docs	Diff analysis → pattern detection
AI sửa đoạn	Chọn text → chọn style	Style preference
So sánh version	Diff view giống Git	Không học, chỉ UI
Giống bài cũ	Chọn bài cũ → AI match style	Template suggestion
Thêm comment	Comment trên từng đoạn	Không học, chỉ collaboration

F.3. Version Control

Mỗi output có version chain:

v1: AI generate (auto)
v2: User sửa tone (manual)
v3: AI sửa CTA (AI edit)
v4: User sửa lại CTA (manual)
v5: User duyệt (final)

→ User có thể:

- Xem bất kỳ version nào
- So sánh 2 version bất kỳ
- Revert về version cũ
- Fork từ version cũ (tạo branch mới)

F.4. “Giống bài cũ” — Style Transfer

```
User chọn "Giống bài cũ" → chọn bài từ history
↓
AI analyze bài cũ:
- Tone analysis
- Structure pattern
- CTA style
- Vocabulary frequency
↓
AI apply pattern vào content mới
↓
User có thể fine-tune
↓
Lưu thành template mới nếu user muốn
```

G. COST & RELIABILITY

G.1. Cost Cap Design

```
Per user, per day:
├─ Max 50 AI calls (CHAT + BUILD)
├─ Max 10 content generations
├─ Max 5 image generations
└─ Max 100K tokens

Per user, per month:
├─ Max 1,500 AI calls
├─ Max 300 content generations
├─ Max 150 image generations
└─ Max 3M tokens

When approaching limit:
├─ 80% → warning: "Bạn đã dùng 80% credit tháng này"
├─ 95% → warning: "Sắp hết credit. Nạp thêm?"
└─ 100% → block: "Credit hết. Vui lòng nạp thêm hoặc đợi tháng sau."
```

G.2. Retry Strategy

```
Exponential backoff:
├─ Attempt 1: immediate
├─ Attempt 2: after 2s
├─ Attempt 3: after 4s
└─ Attempt 4: after 8s → then fail

Circuit breaker:
├─ If API fails 5 times in 1 minute → open circuit
├─ Return fallback: "Service temporarily unavailable"
├─ Retry after 1 minute → half-open
```

└─ If success → close circuit

G.3. Timeout

└─ Web search: 10s
└─ LLM call: 30s
└─ Image generation: 60s
└─ Publish API: 15s
└─ Total job: 5 minutes (then fail)

G.4. Cache Strategy

└─ Research results: 24h TTL
└─ Brand Profile: 1h TTL (rarely change)
└─ Common prompts: 1h TTL
└─ Generated images: 7d TTL
└─ User session: 24h TTL

G.5. Fallback

└─ LLM API down → queue job, notify user "Sẽ thông báo khi xong"
└─ Image gen down → generate text-only, offer "thêm ảnh sau"
└─ Publish API down → queue, retry every 5 min for 1 hour
└─ All services down → maintenance page, email user

H. BUILD PLAN

H.1. 1 Dev — 6 tuần

Scope: Single-channel (Facebook), single content type (caption), no image

Tuần	Task	Output
1	Setup project, auth, DB schema	Login works, DB ready
2	Onboarding flow + Brand Profile	User có thể tạo profile
3	Simple prompt → caption generation	AI viết caption từ prompt
4	Editor + version control	User sửa, lưu version
5	Facebook publish integration	Đăng được lên Facebook
6	Credit system + dashboard	Trừ credit, hiển thị history

Completion: 80% (thiếu: advanced editor, multi-channel, image)

H.2. 2 Dev — 3 tháng

Scope: Facebook + Blog, caption + blog post, basic image

Tháng	Dev A	Dev B
1	Backend: auth, DB, queue, API	Frontend: onboarding, dashboard, editor
2	AI workers: writer, editor, planner	Frontend: review flow, publish UI
3	Integrations: Facebook, WordPress	Image gen integration, testing

Completion: 85% (thiếu: advanced memory, analytics, multi-tenant)

H.3. 5 Dev — 6 tháng

Scope: Full product (all channels, all content types, image, analytics)

Dev	Focus	Months 1-2	Months 3-4	Months 5-6
A	Backend API	Auth, DB, queue	API v2, webhooks	Optimization
B	Frontend	React app, editor	Review, publish UI	Polish, perf
C	AI/ML	Writer, editor, planner	Research, image	Fine-tuning, eval
D	Integrations	Facebook, WordPress	Instagram, TikTok	Testing, docs
E	DevOps/Infra	CI/CD, monitoring	Scaling, security	Reliability

Completion: 90% (thiếu: advanced analytics, A/B testing, enterprise features)

I. RED TEAM — 20 FAILURE MODES

I.1. Failure Analysis

#	Failure	Impact	Likelihood	Mitigation
1	User bỏ onboarding	Không có Brand Profile → output generic	HIGH	Onboarding < 3 phút, có thể skip với warning
2	AI generate content không đúng brand	User thấy vô dụng, bỏ	MEDIUM	Brand Profile bắt buộc, few-shot từ template
3	Editor lag khi AI sửa	User frustrated	MEDIUM	Streaming response, optimistic UI
4	Version control confuse user	User không biết đang xem version nào	MEDIUM	Clear version indicator, auto-save
5	Publish fail, user không biết	Bài không đăng, user nghĩ đã đăng	LOW	Real-time status, push notification
6	Credit hiển thị sai	User tranh cãi, chargeback	LOW	Eventual consistency, real-time counter

Table 11 – continued

#	Failure	Impact	Likelihood	Mitigation
7	Queue backlog	User đợi lâu, bỏ	MEDIUM	Auto-scale workers, priority queue
8	LLM API rate limit	Job fail hàng loạt	HIGH at scale	Multiple provider fallback, circuit breaker
9	Image gen cost spike	Lỗ kinh doanh	MEDIUM	Cost cap, pre-approval for expensive ops
10	Memory leak (Redis)	Server crash	LOW	TTL, memory monitoring, eviction policy
11	User sửa quá nhiều, không bao giờ duyệt	Cost, no revenue	MEDIUM	Max 5 versions, sau đó suggest “start fresh”
12	Brand Profile outdated	Content không còn đúng	MEDIUM	Remind user update every 30 days
13	AI hallucinate research	Content sai fact	HIGH	Restrict research to verified sources, disclaimer
14	Concurrent edit (2 tabs)	Data loss	MEDIUM	Session locking, conflict detection
15	Template bị lỗi	Workflow fail	LOW	Template validation, unit tests
16	User không hiểu “AI sửa”	Sửa không như ý	MEDIUM	Preview before apply, undo
17	Multi-tenant data leak	Security breach	LOW	Row-level security, audit logs
18	Facebook API change	Publish break	MEDIUM	Abstract API layer, quick patch
19	User spam generate	Cost explosion	MEDIUM	Rate limit, cost cap, human review
20	AI suggestion annoying	User tắt notification, miss value	MEDIUM	Limit 1 suggestion/session, easy dismiss

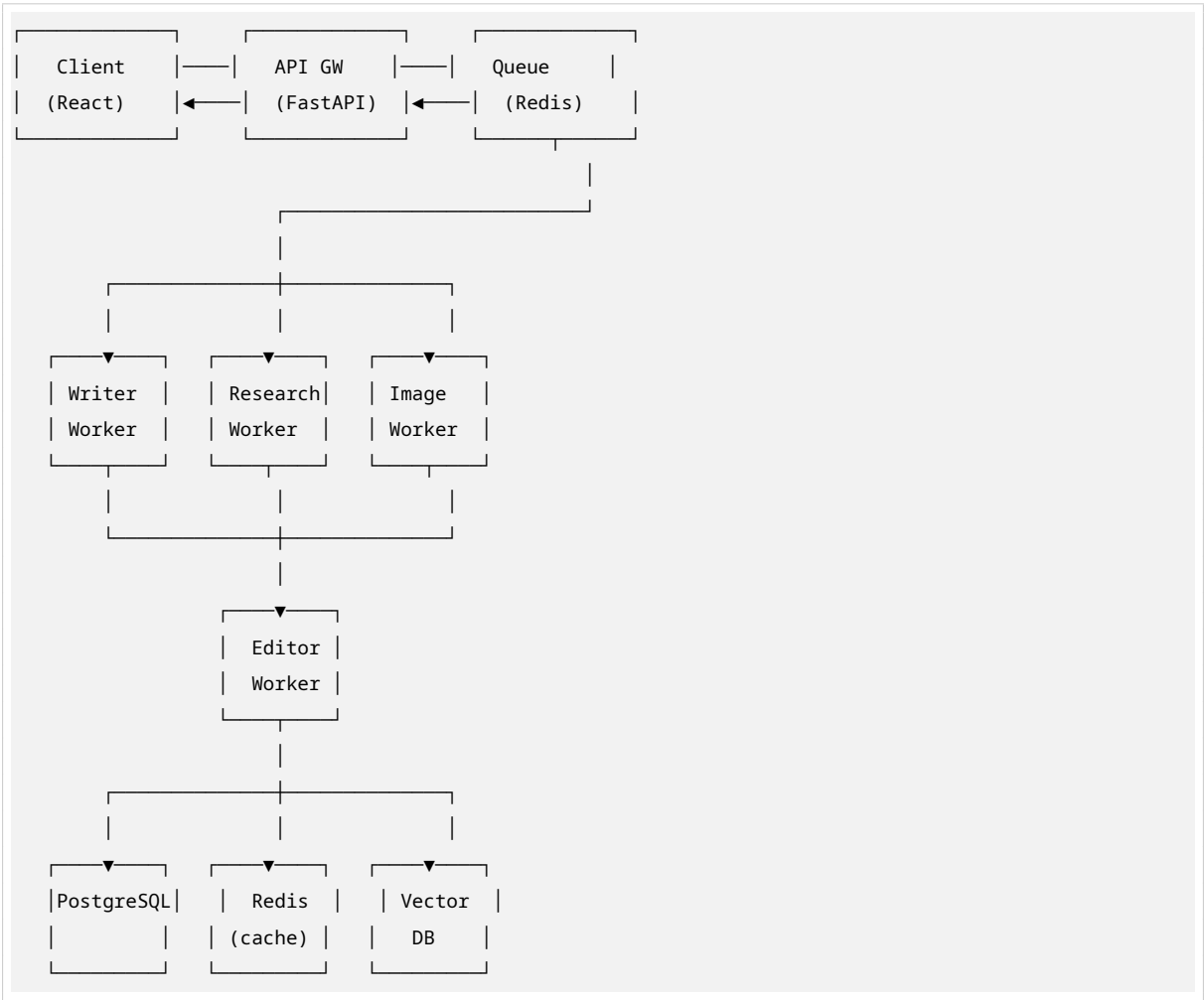
I.2. Self-Critique

Điểm yếu còn lại:

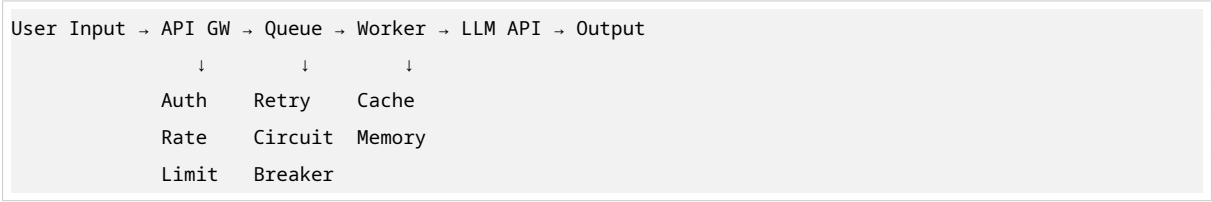
1. **Onboarding vẫn có thể bị bỏ** → cần A/B test length, reward for completion
2. **Editor vẫn phức tạp hơn Google Docs** → cần user testing với non-tech users
3. **Cost per user vẫn cao** → cần optimize prompt, model routing (GPT-4o for complex, GPT-4o-mini for simple)
4. **Scale chưa thiết kế** → cần sharding, read replicas khi > 10K users
5. **Analytics thiếu** → không biết user dùng feature nào nhiều → cần add analytics layer

J. ARCHITECTURE SUMMARY

J.1. System Diagram



J.2. Data Flow



J.3. Key Decisions

Decision	Value
Workspace > Chatbot	User kiểm soát, dễ debug
Predefined Templates > Dynamic Routing	Predictable, dễ test
User-managed Profile > AI-learned	Data quality, no pollution
Explicit Feedback > Implicit Learning	Accurate preferences
Job Queue > Synchronous	Non-blocking, retry, scale
DALL-E API > Self-host	Reliable, no maintenance

Table 12 – continued

Decision	Value
Version Control > Single Draft	User confidence
Cost Cap > Unlimited	Sustainable business

K. MVP vs PRODUCTION

K.1. MVP (6 tuần, 1-2 dev)

Feature	In MVP?
Onboarding (3 câu hỏi)	✓
Brand Profile (basic)	✓
Facebook caption generation	✓
Basic editor (text only)	✓
Version control (last 3)	✓
Facebook publish	✓
Credit system (simple)	✓
Dashboard + history	✓
Blog post	✗
Image generation	✗
Research agent	✗
Multi-channel	✗
Advanced editor (AI inline edit)	✗
Template system	✗
Analytics	✗

K.2. Production (3-6 tháng, 3-5 dev)

Feature	In Production?
All MVP features	✓
Blog post + WordPress	✓
Image generation	✓
Research agent	✓
Instagram + TikTok	✓
Advanced editor	✓
Template system	✓
Analytics dashboard	✓
A/B testing	✓
Team/Agency features	✗ (v2.1)
White-label	✗ (v2.2)

L. QUYẾT ĐỊNH BUILD

L.1. Tổng kết đánh giá

Hạng mục	Điểm	Ghi chú
Product	7/10	Workspace model phù hợp, onboarding cần test
Engineering	7/10	Kiến trúc đơn giản, dễ scale theo phase
AI	6/10	Predefined templates giảm complexity, vẫn cần prompt engineering
Cost	6/10	Cost cap + cache + batch, vẫn cần optimize
Scale	6/10	Queue-based, nhưng chưa thiết kế cho >10K users
Risk	6/10	20 failure modes đã identify, đa số có mitigation
Tổng	6.3/10	

L.2. Quyết định

🟡 BUILD MVP

Lý do:

- Workspace model giải quyết vấn đề core (user kiểm soát, thấy output trước)
- Predefined templates đủ cho 80% use case, không cần dynamic routing phức tạp
- Explicit feedback loop tránh data pollution
- Job queue architecture cho phép scale từng phần

Điều kiện tiên quyết:

1. **Validate onboarding completion rate > 70%** trên 20 beta users trước khi mở rộng
2. **Lock cost per user < \$5/tháng** trước khi mở rộng image generation
3. **Prove retention > 30%** (user quay lại trong 30 ngày) trước khi thêm channels

Không build nếu:

- Onboarding completion < 50%
- User sửa > 5 lần trước khi duyệt (quá nhiều friction)
- Cost per user > \$8/tháng (không profitable)

END OF DOCUMENT